Escuela Superior Politécnica Agropecuaria de Manabí

Informe Portafolio Técnico Final de la Unidad 1

Carrera: Desarrollo de Software

Materia: Programación Orientada a Objetos

Docente: Luiggi Alexander Jalca Saltos

Estudiante: Geovanny Francisco Ríos Rodríguez

Fecha: jueves, 29 de octubre de 2026

Enlace Repositorio: https://github.com/riroguez-espam/POO-2N-ESPAM-GFRR

1. Objetivo Pedagógico del Portafolio

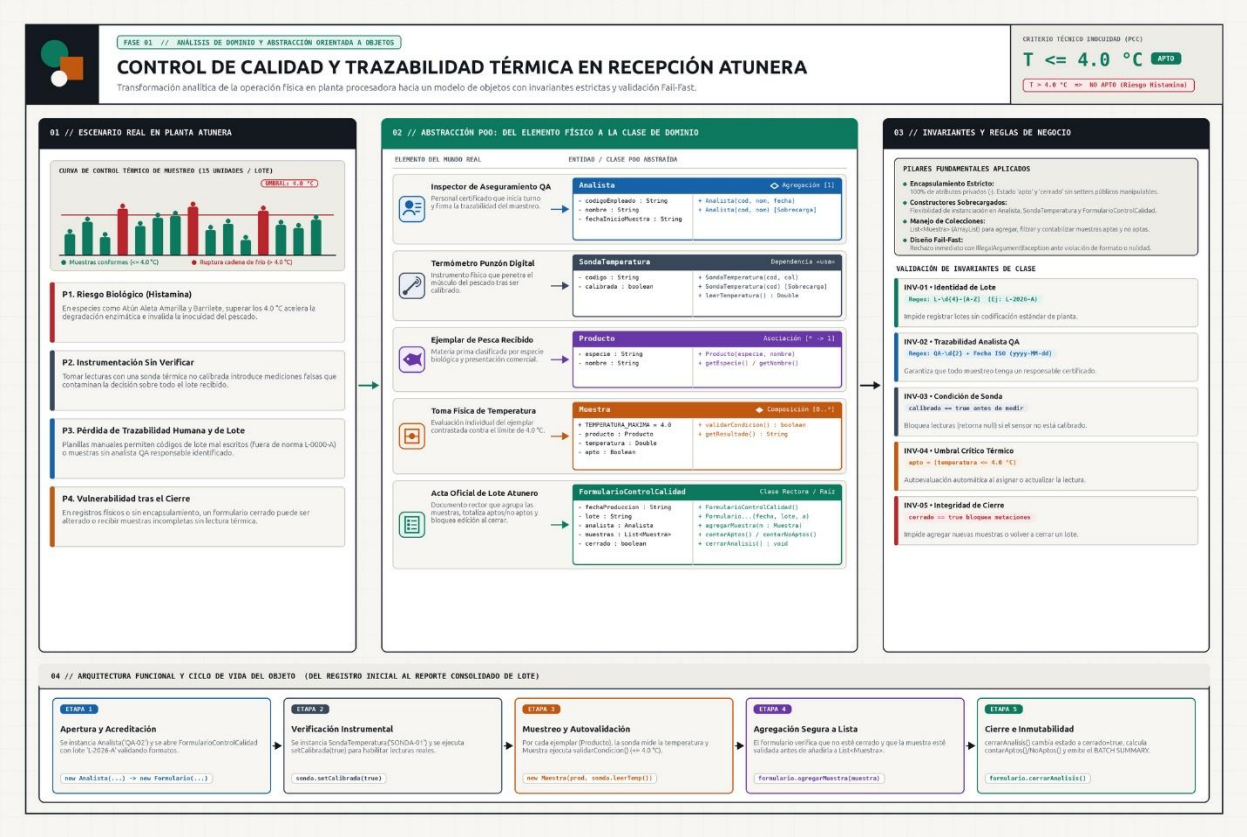
El presente Portafolio Técnico de Ingeniería de Software tiene como propósito evidenciar de manera rigurosa y práctica la aplicación de los fundamentos de la Programación Orientada a Objetos (abstracción, encapsulamiento, visibilidad, constructores sobrecargados, validación de invariantes de clase y manejo de colecciones dinámicas) en la resolución de una problemática real de la industria pesquera: el sistema de Control de Calidad y Trazabilidad Térmica en una Planta Procesadora y Recepción de Pesca Atunera.

En el contexto operativo de la recepción atunera, el aseguramiento de la inocuidad alimentaria depende estrictamente del control de la cadena de frío. Especies pelágicas como el atún (albacora o barrilete) son altamente susceptibles a la formación de histamina por descomposición enzimática si su temperatura muscular interna supera el límite crítico de 4.0 °C. Modelar este proceso mediante el paradigma orientado a objetos permite transformar reglas sanitarias y operativas del mundo físico en contratos de software inquebrantables, evitando que se registren lotes con nomenclatura inválida, mediciones provenientes de sondas térmicas sin calibrar, muestras huérfanas de analista responsable o alteraciones posteriores al cierre oficial del acta de inspección.

Asimismo, este portafolio demuestra la universalidad, adaptabilidad y portabilidad del diseño orientado a objetos mediante la migración íntegra de la arquitectura desde Java hacia TypeScript. Esta práctica comprueba que un modelo de dominio bien concebido —sustentado en el principio Fail-Fast y en invariantes de clase bien delimitadas— trasciende las particularidades sintácticas de un lenguaje de programación específico, manteniendo intacta su lógica de negocio, su seguridad de estado y su comportamiento verificable en distintas plataformas tecnológicas.

2. Evidencias Gráficas Comentadas por Fase

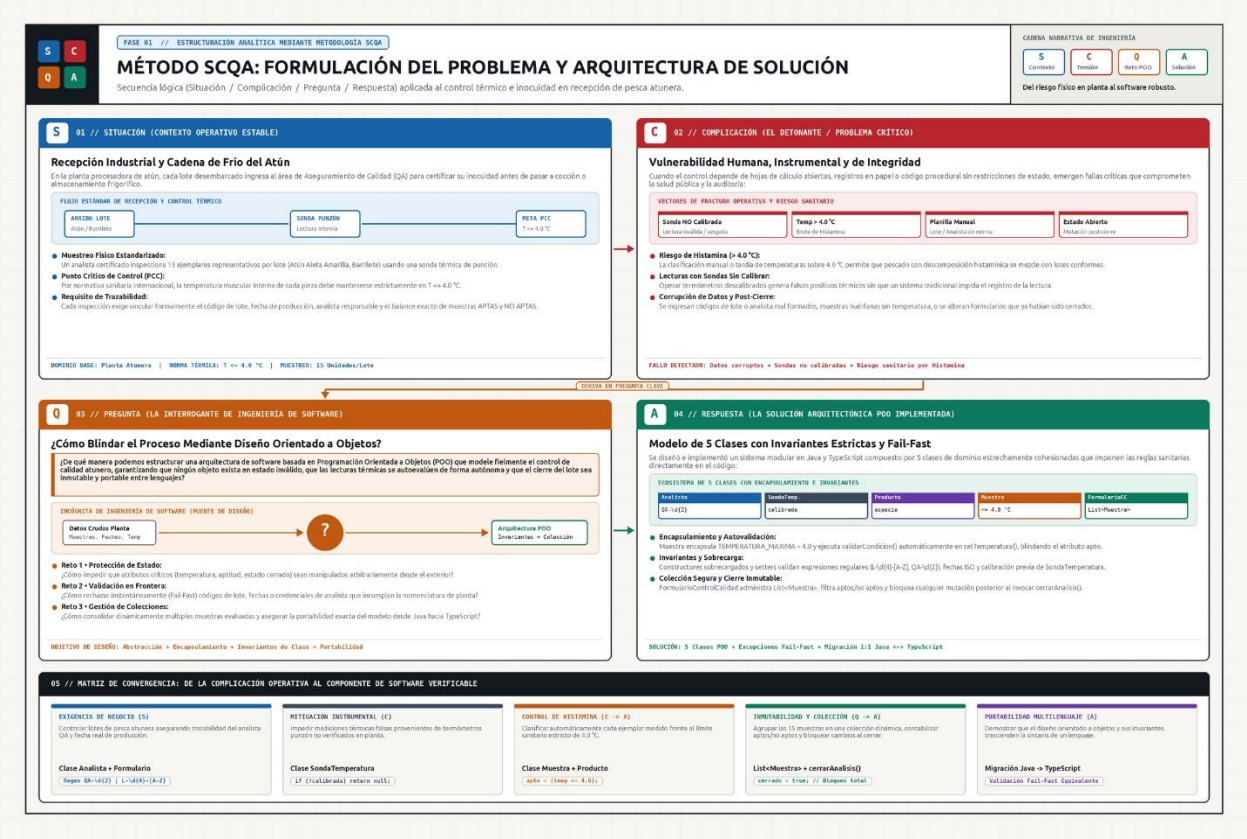
2.1. Fase 01: Análisis del Problema



La lámina de análisis del problema ilustra el proceso de abstracción ejecutado para trasladar la operación física del área de Aseguramiento de Calidad (QA) de la planta atunera hacia el paradigma orientado a objetos. En primera instancia, se identificaron cuatro problemáticas operativas críticas del entorno real: el riesgo biológico de generación de histamina cuando el pescado supera los 4.0 °C, la incertidumbre instrumental derivada de tomar lecturas con termómetros punzón no calibrados, la pérdida de trazabilidad cuando se emplean planillas manuales sin validación de formato, y la vulnerabilidad de los registros frente a modificaciones no autorizadas una vez finalizado el muestreo.

A partir de este diagnóstico, se abstrajeron cinco entidades fundamentales del dominio descartando detalles irrelevantes y encapsulando únicamente el estado y el comportamiento esenciales para el negocio: el inspector humano certificado se modeló en la clase Analista; el instrumento físico de punción se representó en la clase SondaTemperatura; la materia prima evaluada se definió en la

clase Producto; el acto físico de medición individual y su veredicto térmico se encapsularon en la clase Muestra; y el acta oficial que consolida las quince muestras del lote se estructuró en la clase rectora FormularioControlCalidad.

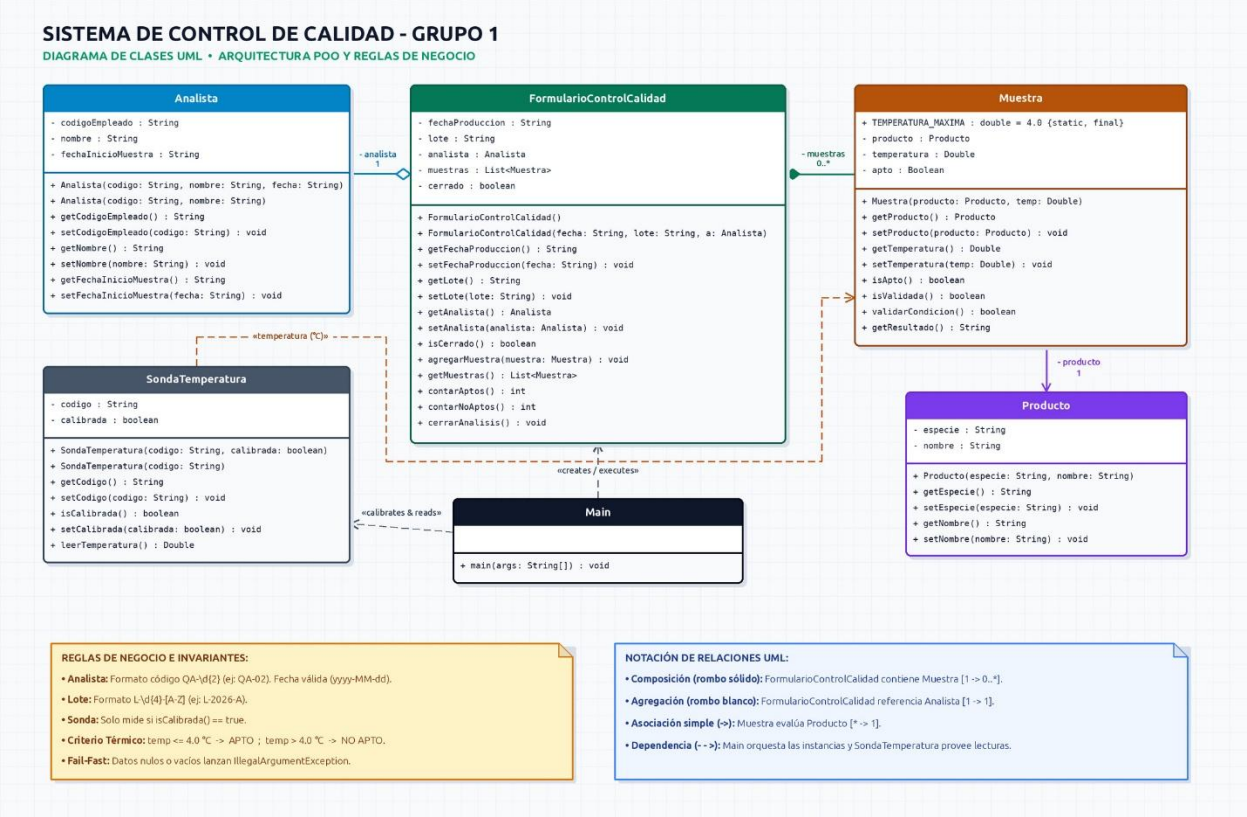


Para estructurar formalmente la presentación de la solución de ingeniería, se aplicó la metodología analítica SCQA (Situación, Complicación, Pregunta y Respuesta). En la Situación (S), se establece el contexto operativo estándar de la planta procesadora, donde cada lote desembarcado requiere el muestreo físico de 15 ejemplares verificando que su temperatura interna se mantenga menor o igual a 4.0 °C bajo la supervisión de un analista acreditado.

La Complicación (C) surge cuando este control se apoya en registros manuales o software sin protección de estado, permitiendo el ingreso de códigos de lote corruptos, lecturas tomadas con sondas descalibradas, clasificación errónea de ejemplares por encima de 4.0 °C y alteración de lotes ya cerrados. Esto conduce a la Pregunta central de ingeniería (Q): ¿cómo estructurar una arquitectura orientada a objetos que blinde la integridad del control de calidad atunero, garantizando que ningún objeto nazca en estado inválido y que el modelo sea portable entre

lenguajes? Finalmente, la Respuesta (A) se materializa en el diseño de las cinco clases de dominio con atributos privados, constructores sobrecargados, colecciones tipadas y validación Fail-Fast tanto en Java como en TypeScript.

2.2. Fase 02: Diagrama de Clases UML



El diagrama de clases UML documenta la arquitectura estática del módulo de Control de Calidad aplicando estrictamente la nomenclatura estándar de visibilidad y encapsulamiento. Todos los atributos de instancia en las cinco clases de dominio llevan el modificador de visibilidad privado (-), impidiendo el acceso o la mutación directa desde clases externas. Por su parte, los constructores, métodos de acceso (getters y setters validados) y métodos de negocio se exponen con visibilidad pública (+), destacando además la constante estática y final TEMPERATURA_MAXIMA = 4.0 en la clase Muestra como regla invariante del dominio.

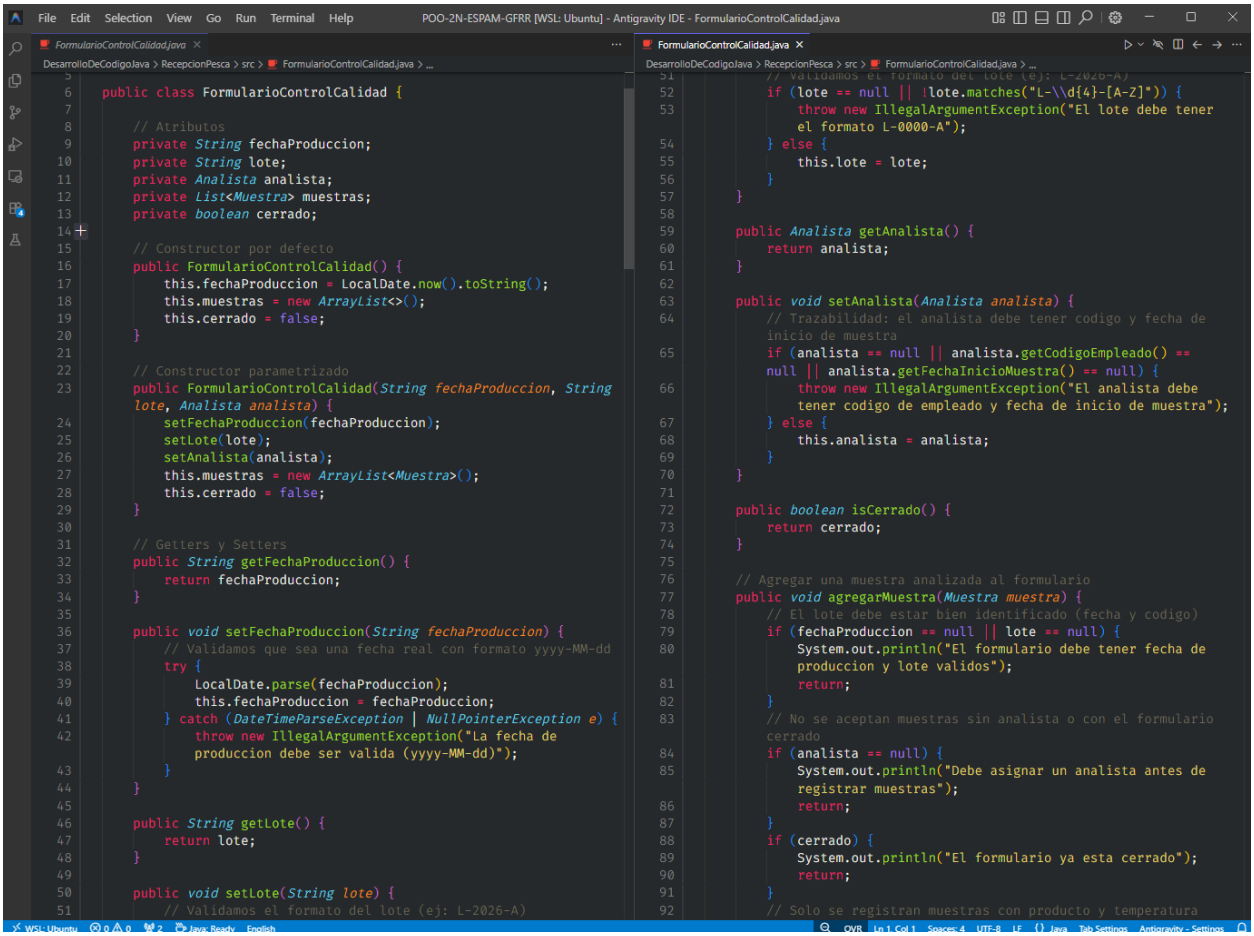
En cuanto a las relaciones estructurales y multiplicidades, la clase central FormularioControlCalidad mantiene una relación de Composición (rombo sólido) con multiplicidad de 1 a 0..* hacia la clase Muestra, dado que las muestras registradas forman parte

integral del ciclo de vida del formulario de inspección a través de la colección List<Muestra>. Asimismo, FormularioControlCalidad establece una relación de Agregación (rombo vacío) con multiplicidad 1 a 1 hacia la clase Analista, reflejando que todo formulario requiere obligatoriamente un analista responsable asignado, pero el analista existe como entidad independiente en la planta.

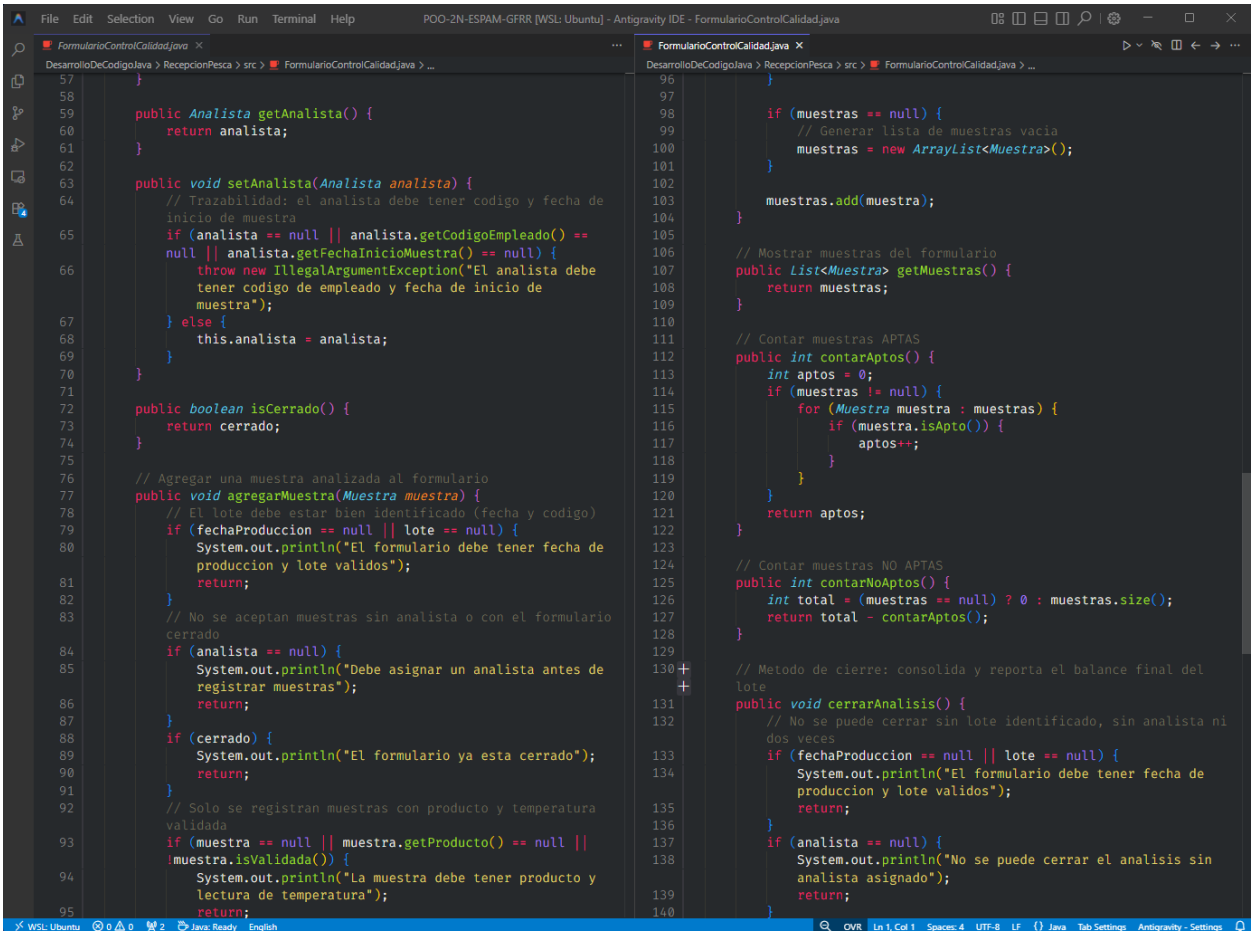
Adicionalmente, la clase Muestra presenta una Asociación directa navegable (* a 1) hacia la clase Producto, indicando que múltiples tomas de temperatura evalúan un producto pesquero específico (como Atún Albacora). Finalmente, se modelan las relaciones de Dependencia (líneas discontinuas) desde la clase orquestadora Main hacia FormularioControlCalidad y SondaTemperatura, así como el suministro de lecturas térmicas desde SondaTemperatura hacia Muestra, evidenciando también la sobrecarga de constructores en Analista, SondaTemperatura y FormularioControlCalidad.

2.3. Fase 03: Desarrollo de Código en Java

La implementación en Java se desarrolló aplicando validación defensiva de tipo Fail-Fast (fallar rápido), asegurando que cualquier intento de instanciar un objeto o modificar un atributo con datos fuera de norma dispare inmediatamente una excepción IllegalArgumentException antes de corromper el estado en memoria. La clase de negocio más completa del módulo es FormularioControlCalidad.java, la cual coordina la trazabilidad del lote, el analista, la colección de muestras y el cierre inmutable del análisis. Para garantizar que las capturas de código en el documento mantengan un tamaño compacto y una nitidez óptima, se divide la evidencia de esta clase en dos fragmentos específicos.



En este primer fragmento de FormularioControlCalidad.java (líneas 6 a 74) se observa el encapsulamiento estricto de los cinco atributos privados: fechaProduccion, lote, analista, la colección muestras de tipo List<Muestra> y la bandera booleana cerrado. Se implementan dos constructores sobrecargados: un constructor por defecto que inicializa la fecha al día actual y la lista vacía, y un constructor parametrizado que delega la asignación a los métodos setters para forzar el cumplimiento de las invariantes desde el nacimiento del objeto. En setFechaProduccion() se valida que la cadena corresponda a una fecha ISO real mediante LocalDate.parse(); en setLote() se exige el cumplimiento estricto de la expresión regular L-\d{4}-[A-Z] (por ejemplo, L-2026-A); y en setAnalista() se verifica que el objeto Analista no sea nulo y posea código de empleado y fecha de inicio válidos. Cabe destacar que el atributo cerrado carece de método setter público, exponiendo únicamente el método de consulta isCerrado().



```
}

public Analista getAnalista() {
    return analista;
}

public void setAnalista(Analista analista) {
    // Trazabilidad: el analista debe tener codigo y fecha de inicio de muestra
    if (analista == null || analista.getCodigoEmpleado() == null || analista.getFechaInicioMuestra() == null) {
        throw new IllegalArgumentException("El analista debe tener codigo de empleado y fecha de inicio de muestra");
    } else {
        this.analista = analista;
    }
}

public boolean isCerrado() {
    return cerrado;
}

// Agregar una muestra analizada al formulario
public void agregarMuestra(Muestra muestra) {
    // El lote debe estar bien identificado (fecha y codigo)
    if (fechaProduccion == null || lote == null) {
        System.out.println("El formulario debe tener fecha de produccion y lote validos");
        return;
    }
    // No se aceptan muestras sin analista o con el formulario cerrado
    if (analista == null) {
        System.out.println("Debe asignar un analista antes de registrar muestras");
        return;
    }
    if (cerrado) {
        System.out.println("El formulario ya esta cerrado");
        return;
    }
    // Solo se registran muestras con producto y temperatura validada
    if (muestra == null || muestra.getProducto() == null || !muestra.isValidada()) {
        System.out.println("La muestra debe tener producto y lectura de temperatura");
        return;
    }

    if (muestras == null) {
        // Generar lista de muestras vacia
        muestras = new ArrayList<Muestra>();
    }

    muestras.add(muestra);
}

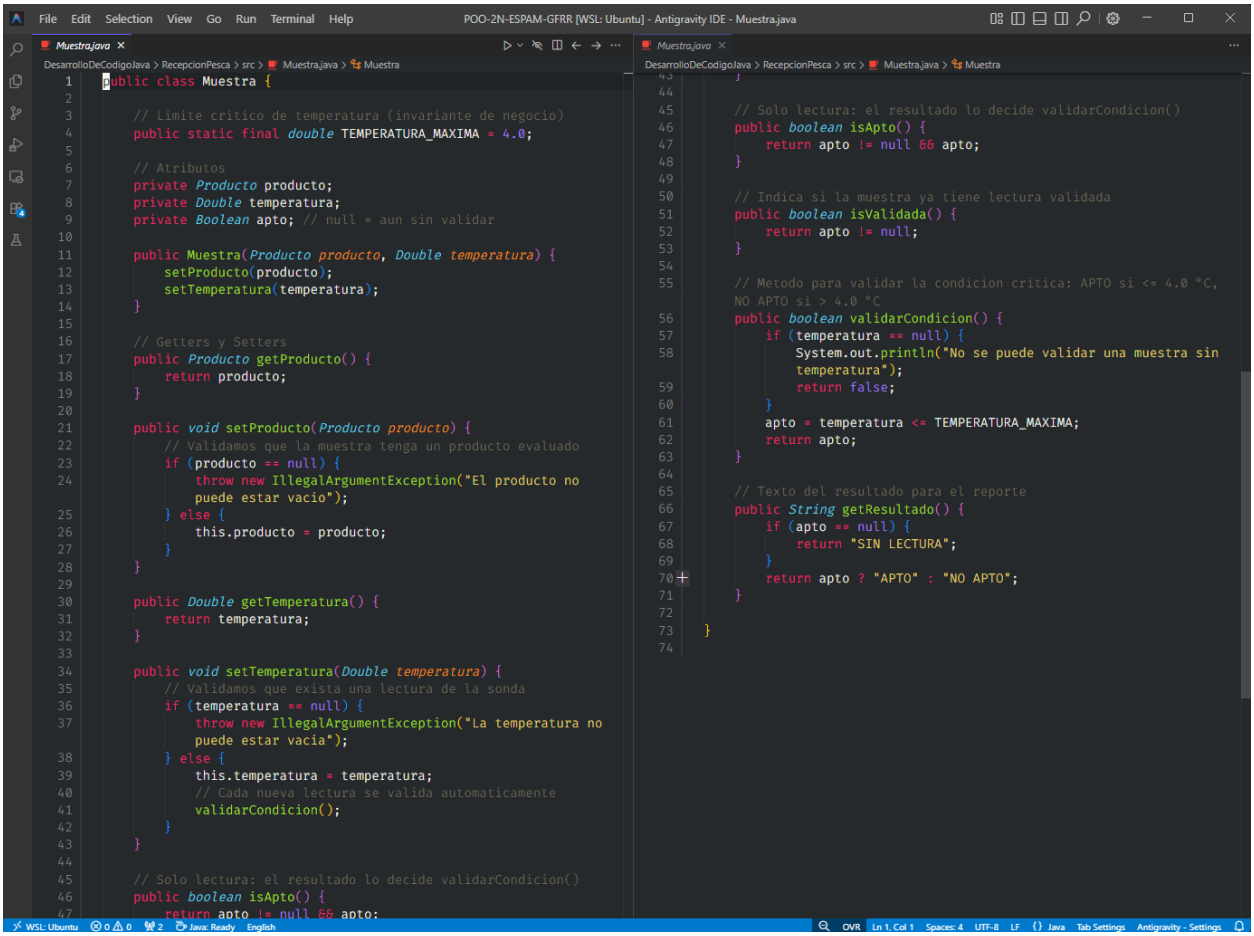
// Mostrar muestras del formulario
public List<Muestra> getMuestras() {
    return muestras;
}

// Contar muestras APTAS
public int contarAptos() {
    int aptos = 0;
    if (muestras != null) {
        for (Muestra muestra : muestras) {
            if (muestra.isApto()) {
                aptos++;
            }
        }
    }
    return aptos;
}

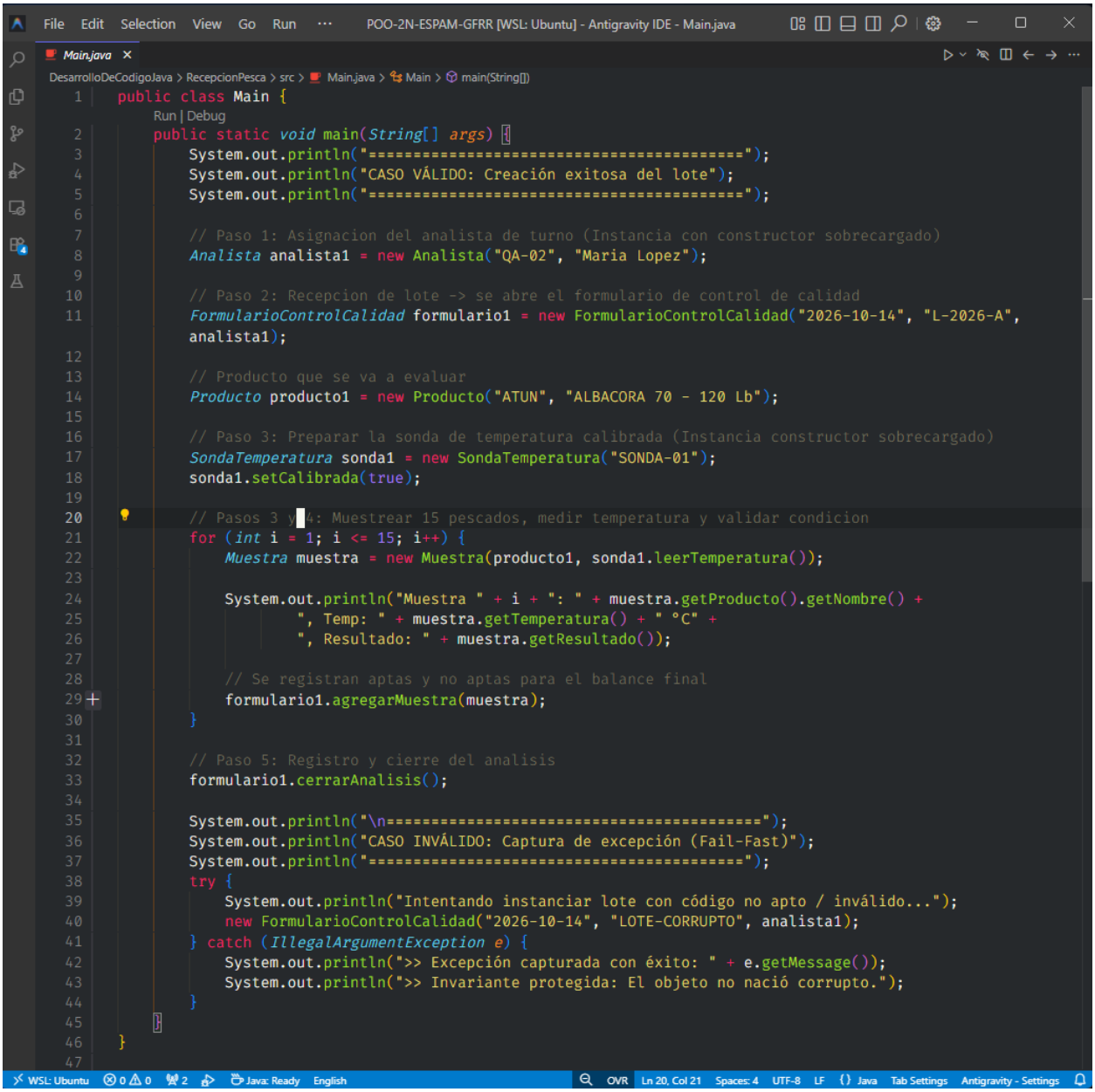
// Contar muestras NO APTAS
public int contarNoAptos() {
    int total = (muestras == null) ? 0 : muestras.size();
    return total - contarAptos();
}

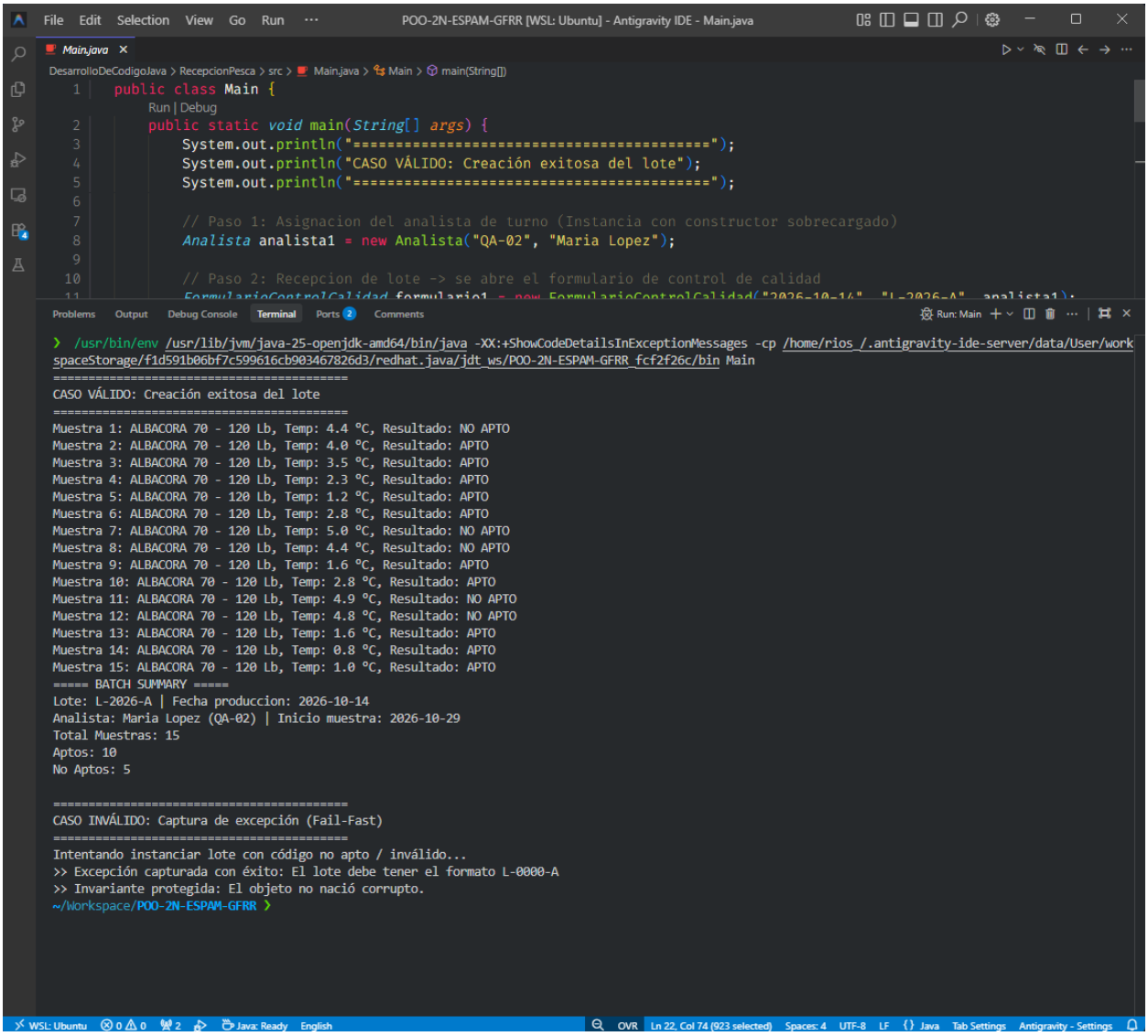
// Metodo de cierre: consolida y reporta el balance final del lote
public void cerrarAnalisis() {
    // No se puede cerrar sin lote identificado, sin analista ni dos veces
    if (fechaProduccion == null || lote == null) {
        System.out.println("El formulario debe tener fecha de produccion y lote validos");
        return;
    }
    if (analista == null) {
        System.out.println("No se puede cerrar el analisis sin analista asignado");
        return;
    }
```

En el segundo fragmento de FormularioControlCalidad.java (líneas 76 a 158) se implementa la lógica de negocio y el manejo de la colección ArrayList<Muestra>. El método agregarMuestra() actúa como guardián de estado: verifica que el formulario posea lote y analista válidos, comprueba que el análisis no haya sido cerrado previamente y exige que la muestra recibida tenga un producto asociado y una lectura térmica ya validada (muestra.isValidada()). Por su parte, los métodos contarAptos() y contarNoAptos() recorren la colección evaluando el estado encapsulado de cada muestra, mientras que cerrarAnalisis() cambia irreversiblemente el estado interno a cerrado = true e imprime el reporte consolidado (BATCH SUMMARY), bloqueando cualquier adición o cierre posterior.



Como complemento fundamental de las invariantes del dominio, el fragmento de Muestra.java (líneas 1 a 64) evidencia cómo se protege el criterio sanitario crítico de la planta atunera. La constante TEMPERATURA_MAXIMA se fija en 4.0 °C y el atributo apto se mantiene privado y sin método setter. Cuando el constructor o el método setTemperatura() reciben la lectura proveniente de la sonda, primero verifican que no sea nula (lo que ocurriría si la sonda no estuviera calibrada) y acto seguido invocan automáticamente a validarCondicion(), asignando true si la temperatura es menor o igual a 4.0 °C (APTO) o false si supera dicho umbral (NO APTO).


File Edit Selection View Go Run ··· POO-2N-ESPAM-GFRR [WSL: Ubuntu] - Antigravity IDE - Main.java
Main.java
DesarrolloDeCodigoJava > RecepcionPesca > src > Main.java > Main > main(String[])
public class Main {
Run | Debug
public static void main(String[] args) {
System.out.println("==");
System.out.println("CASO VÁLIDO: Creación exitosa del lote");
System.out.println("==");
// Paso 1: Asignacion del analista de turno (Instancia con constructor sobrecargado)
Analista analista1 = new Analista("QA-02", "Maria Lopez");
// Paso 2: Recepcion de lote -> se abre el formulario de control de calidad
FormularioControlCalidad formulario1 = new FormularioControlCalidad("2026-10-14", "L-2026-A",
analista1);
// Producto que se va a evaluar
Producto producto1 = new Producto("ATUN", "ALBACORA 70 - 120 Lb");
// Paso 3: Preparar la sonda de temperatura calibrada (Instancia constructor sobrecargado)
SondaTemperatura sonda1 = new SondaTemperatura("SONDA-01");
sonda1.setCalibrada(true);
// Pasos 3 y 4: Muestrear 15 pescados, medir temperatura y validar condicion
for (int i = 1; i <= 15; i++) {
Muestra muestra = new Muestra(producto1, sonda1.leerTemperatura());
System.out.println("Muestra " + i + ": " + muestra.getProducto().getNombre() +
", Temp: " + muestra.getTemperatura() + " °C" +
", Resultado: " + muestra.getResultado());
// Se registran aptas y no aptas para el balance final
formulario1.agregarMuestra(muestra);
}
// Paso 5: Registro y cierre del analisis
formulario1.cerrarAnalisis();
System.out.println("\n==");
System.out.println("CASO INVÁLIDO: Captura de excepción (Fail-Fast)");
System.out.println("==");
try {
System.out.println("Intentando instanciar lote con código no apto / inválido...");
new FormularioControlCalidad("2026-10-14", "LOTE-CORRUPTO", analista1);
} catch (IllegalArgumentException e) {
System.out.println(">> Excepción capturada con éxito: " + e.getMessage());
System.out.println(">> Invariante protegida: El objeto no nació corrupto.");
}
}
}
WSL: Ubuntu 0 0 2 Java: Ready English OVR Ln 20, Col 21 Spaces: 4 UTF-8 LF Java Tab Settings Antigravity - Settings

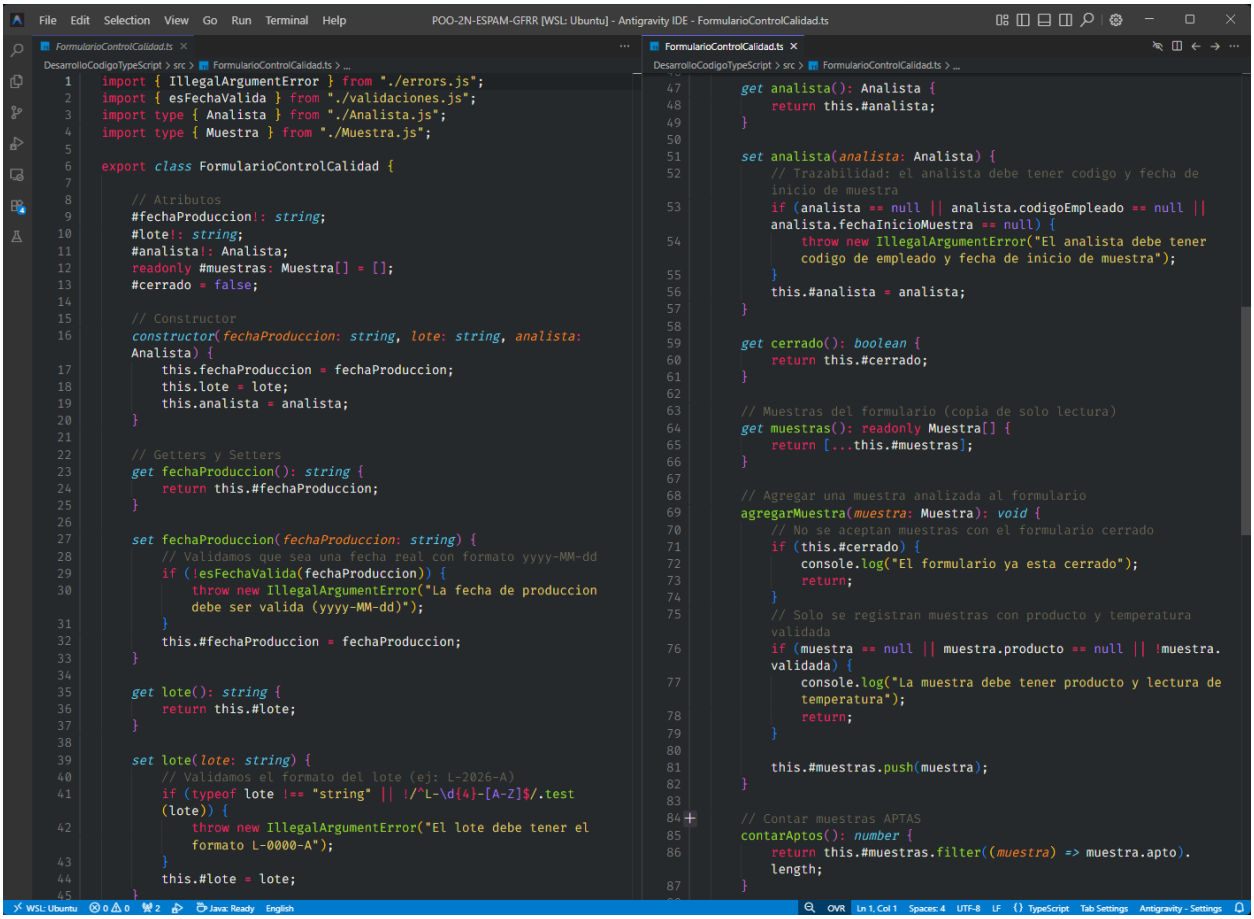


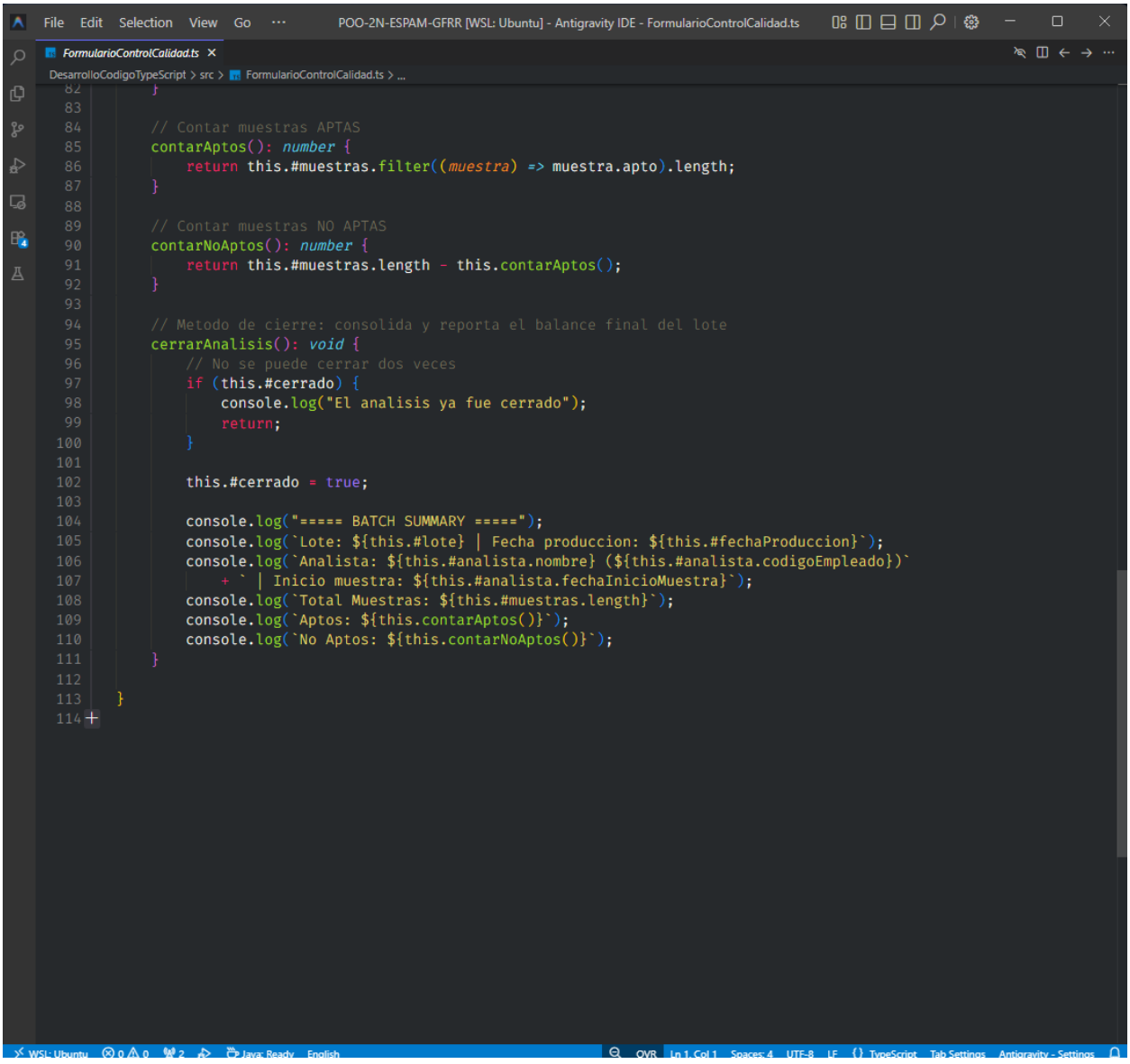
En la clase Main.java (líneas 1 a 46) y su respectiva ejecución en consola se comprueba el funcionamiento integral del sistema en dos escenarios. En el Caso Válido, se instancia al analista 'QA-02' usando su constructor sobrecargado de dos parámetros, se abre el formulario para el lote 'L-2026-A', se calibra la sonda 'SONDA-01' y se ejecutan 15 iteraciones de muestreo sobre el producto 'ALBACORA 70 - 120 Lb', culminando con la emisión del resumen del lote. En el Caso Inválido, se intenta instanciar un formulario con el identificador 'LOTE-CORRUPTO' dentro de un bloque try-catch, demostrando el disparo controlado de IllegalArgumentException con el mensaje 'El lote debe tener el formato L-0000-A', garantizando así que ningún objeto corrupto llegue a existir en memoria.

2.4. Fase 04: Desarrollo de Código en Lenguaje Escogido (TypeScript)

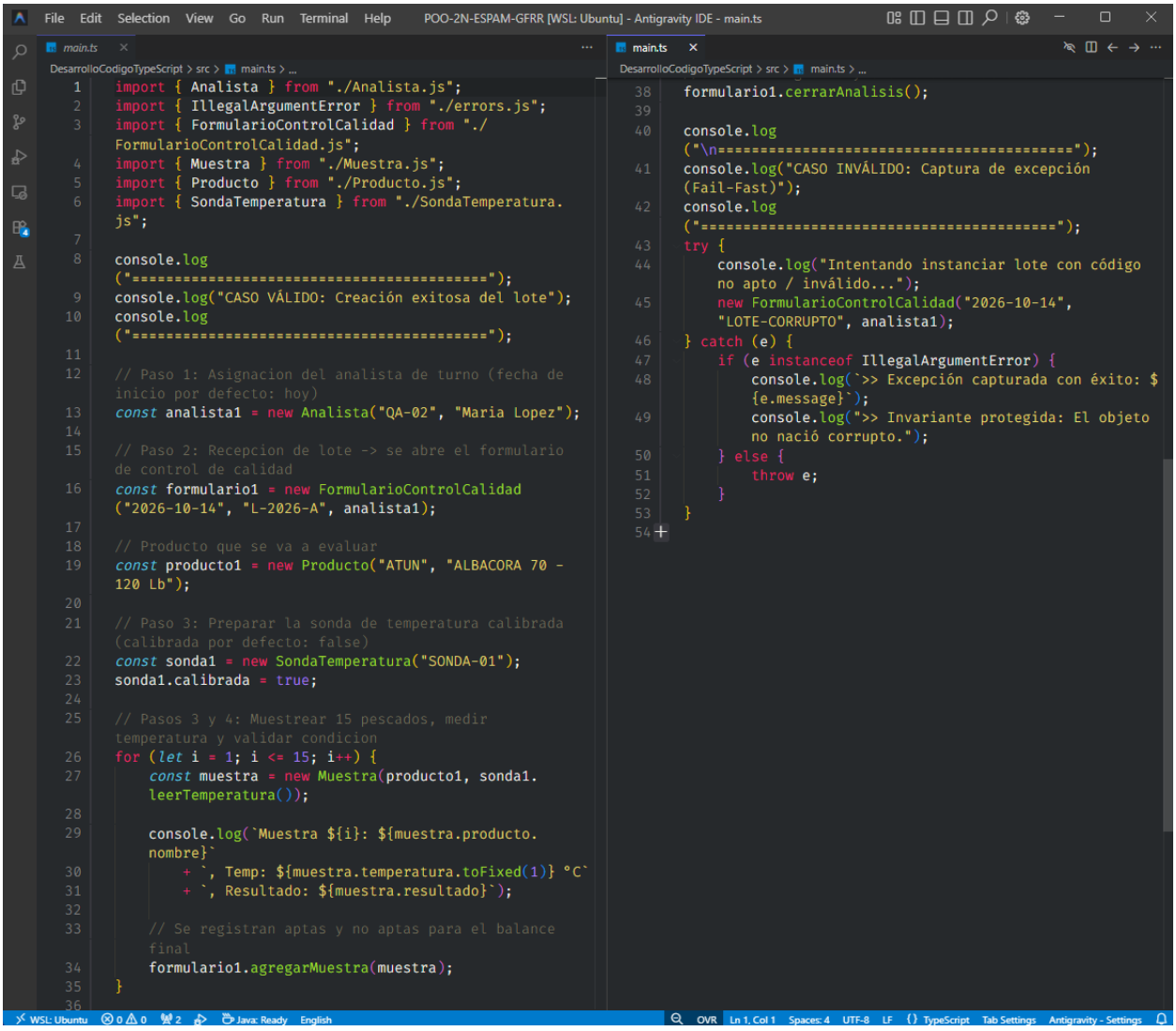
Para la práctica de migración a un segundo lenguaje autorizado se seleccionó TypeScript (ejecutado sobre el entorno Node.js). La elección de TypeScript se justifica técnicamente porque combina la flexibilidad y ubicuidad del ecosistema moderno de JavaScript con un sistema de tipado estático estricto en tiempo de compilación y soporte nativo para los estándares más recientes de ECMAScript. Esto permite realizar una traducción idiomática de alto nivel desde Java, conservando la seguridad de tipos, el ocultamiento real de estado en tiempo de ejecución y el manejo estructurado de excepciones.

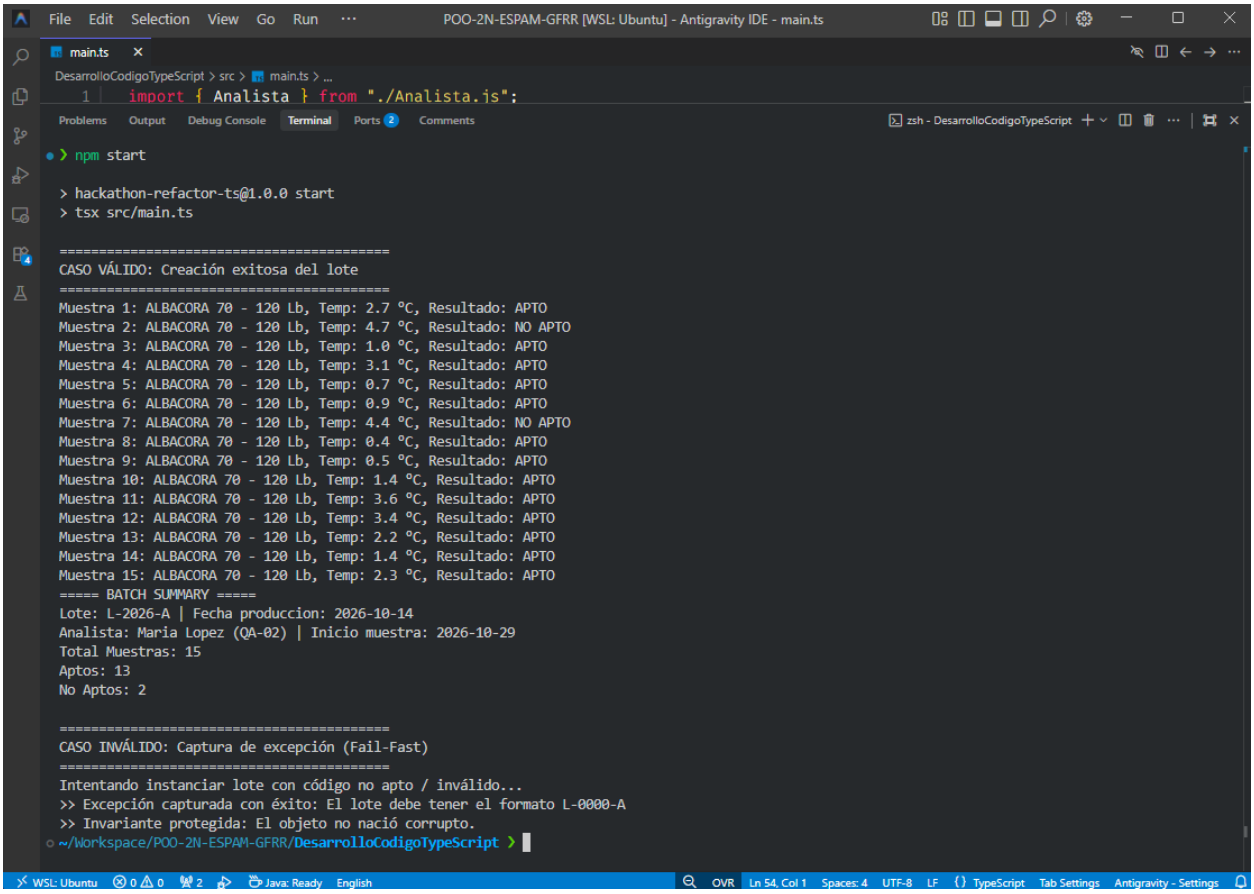
En cuanto a las equivalencias de sintaxis y diseño con respecto a Java, se aplicaron cuatro adaptaciones idiomáticas clave. Primero, los atributos privados de Java (private) se migraron a campos privados duros de ECMAScript mediante el prefijo numeral (#fechaProduccion, #lote, #analista, #muestras, #cerrado), los cuales garantizan privacidad real tanto en compilación como en ejecución. Segundo, la sobrecarga de constructores de Java en clases como Analista y SondaTemperatura se expresó de forma idiomática en TypeScript mediante parámetros con valores por defecto en el constructor (por ejemplo, fechaInicioMuestra: string = hoyISO() y calibrada: boolean = false). Tercero, los métodos tradicionales getX() y setX() de Java se transformaron en propiedades accesoras nativas get y set de TypeScript, permitiendo validar invariantes de forma transparente al asignar propiedades. Cuarto, la colección List<Muestra> de Java se implementó mediante un arreglo privado de solo lectura (readonly #muestras: Muestra[] = []) cuyos elementos se exponen mediante una copia superficial inmutable (return [...this.#muestras]) y se contabilizan funcionalmente con el método filter().


File Edit Selection View Go Run Terminal Help
POO-2N-ESPAM-GFRR [WSL: Ubuntu] - Antigravity IDE - FormularioControlCalidad.ts
FormularioControlCalidad.ts
DesarrolloCodigoTypeScript > src > FormularioControlCalidad.ts > ...
import { IllegalArgumentError } from "./errors.js";
import { esFechaValida } from "./validaciones.js";
import type { Analista } from "./Analista.js";
import type { Muestra } from "./Muestra.js";
export class FormularioControlCalidad {
// Atributos
#fechaProduccion!: string;
#lote!: string;
#analista!: Analista;
readonly #muestras: Muestra[] = [];
#cerrado = false;
// Constructor
constructor(fechaProduccion: string, lote: string, analista: Analista) {
this.fechaProduccion = fechaProduccion;
this.lote = lote;
this.analista = analista;
}
// Getters y Setters
get fechaProduccion(): string {
return this.#fechaProduccion;
}
set fechaProduccion(fechaProduccion: string) {
// Validamos que sea una fecha real con formato yyyy-MM-dd
if (!esFechaValida(fechaProduccion)) {
throw new IllegalArgumentError("La fecha de produccion debe ser valida (yyyy-MM-dd)");
}
this.#fechaProduccion = fechaProduccion;
}
get lote(): string {
return this.#lote;
}
set lote(lote: string) {
// Validamos el formato del lote (ej: L-2026-A)
if (typeof lote !== "string" || !/^L-\d{4}-[A-Z]$/.test(lote)) {
throw new IllegalArgumentError("El lote debe tener el formato L-0000-A");
}
this.#lote = lote;
}
get analista(): Analista {
return this.#analista;
}
set analista(analista: Analista) {
// Trazabilidad: el analista debe tener codigo y fecha de inicio de muestra
if (analista == null || analista.codigoEmpleado == null || analista.fechaInicioMuestra == null) {
throw new IllegalArgumentError("El analista debe tener codigo de empleado y fecha de inicio de muestra");
}
this.#analista = analista;
}
get cerrado(): boolean {
return this.#cerrado;
}
// Muestras del formulario (copia de solo lectura)
get muestras(): readonly Muestra[] {
return [...this.#muestras];
}
// Agregar una muestra analizada al formulario
agregarMuestra(muestra: Muestra): void {
// No se aceptan muestras con el formulario cerrado
if (this.#cerrado) {
console.log("El formulario ya esta cerrado");
return;
}
// Solo se registran muestras con producto y temperatura validada
if (muestra == null || muestra.producto == null || !muestra.validada) {
console.log("La muestra debe tener producto y lectura de temperatura");
return;
}
this.#muestras.push(muestra);
}
// Contar muestras APTAS
contarAptos(): number {
return this.#muestras.filter((muestra) => muestra.apto).length;
}
WSL: Ubuntu
Java: Ready
English
Ln 1, Col 1
Spaces: 4
UTF-8
LF
TypeScript
Tab Settings
Antigravity - Settings



En los dos fragmentos de FormularioControlCalidad.ts (líneas 6 a 66 y 68 a 113) se evidencia la fidelidad del modelo migrado. Los setters validan exactamente las mismas reglas de negocio que en Java, lanzando una instancia de IllegalArgumentError (definida en errors.ts como subclase de Error equivalente a IllegalArgumentException) cuando la fecha no supera la verificación calendárica de esFechaValida() o cuando el código de lote incumple el patrón /^L-\d{4}-[A-Z]$/. Asimismo, el método agregarMuestra() protege la colección frente a formularios cerrados o muestras sin lectura validada, y contarAptos() aprovecha la programación funcional idiomática de TypeScript (this.#muestras.filter((muestra) => muestra.apto).length).


File Edit Selection View Go Run Terminal Help
POO-2N-ESPAM-GFRR [WSL: Ubuntu] - Antigravity IDE - main.ts
main.ts
DesarrolloCodigoTypeScript > src > main.ts > ...
import { Analista } from "./Analista.js";
import { IllegalArgumentError } from "./errors.js";
import { FormularioControlCalidad } from "./FormularioControlCalidad.js";
import { Muestra } from "./Muestra.js";
import { Producto } from "./Producto.js";
import { SondaTemperatura } from "./SondaTemperatura.js";
console.log("==========");
console.log("CASO VÁLIDO: Creación exitosa del lote");
console.log("==========");
// Paso 1: Asignacion del analista de turno (fecha de inicio por defecto: hoy)
const analista1 = new Analista("QA-02", "Maria Lopez");
// Paso 2: Recepcion de lote -> se abre el formulario de control de calidad
const formulario1 = new FormularioControlCalidad("2026-10-14", "L-2026-A", analista1);
// Producto que se va a evaluar
const producto1 = new Producto("ATUN", "ALBACORA 70 - 120 Lb");
// Paso 3: Preparar la sonda de temperatura calibrada (calibrada por defecto: false)
const sonda1 = new SondaTemperatura("SONDA-01");
sonda1.calibrada = true;
// Pasos 3 y 4: Muestrear 15 pescados, medir temperatura y validar condicion
for (let i = 1; i <= 15; i++) {
const muestra = new Muestra(producto1, sonda1.leerTemperatura());
console.log(`Muestra ${i}: ${muestra.producto.nombre}`
+ `, Temp: ${muestra.temperatura.toFixed(1)} °C`
+ `, Resultado: ${muestra.resultado}`);
// Se registran aptas y no aptas para el balance final
formulario1.agregarMuestra(muestra);
}
main.ts
DesarrolloCodigoTypeScript > src > main.ts > ...
formulario1.cerrarAnalisis();
console.log("\n==========");
console.log("CASO INVÁLIDO: Captura de excepción (Fail-Fast)");
console.log("==========");
try {
console.log("Intentando instanciar lote con código no apto / inválido...");
new FormularioControlCalidad("2026-10-14", "LOTE-CORRUPTO", analista1);
} catch (e) {
if (e instanceof IllegalArgumentError) {
console.log(`>> Excepción capturada con éxito: ${e.message}`);
console.log(">> Invariante protegida: El objeto no nació corrupto.");
} else {
throw e;
}
}
WSL: Ubuntu
Java: Ready
English
Ln 1, Col 1
Spaces: 4
UTF-8
LF
TypeScript
Tab Settings
Antigravity - Settings



Finalmente, la captura del archivo main.ts (líneas 1 a 53) y su salida en consola al ejecutar npm run start demuestran una equivalencia funcional exacta con respecto a la versión en Java. El script instancia las mismas entidades de negocio, ejecuta el muestreo térmico de los 15 ejemplares de albacora, consolida el balance de muestras aptas y no aptas al cerrar el análisis e intercepta de manera controlada el intento de crear el lote inválido 'LOTE-CORRUPTO', confirmando que las invariantes de clase operan con idéntica rigurosidad en ambos lenguajes.

3. Conclusión Técnica

El desarrollo integrador del sistema de Control de Calidad para la planta procesadora de atún demuestra que la Programación Orientada a Objetos constituye mucho más que una forma de organizar archivos de código: es un mecanismo formal de ingeniería para modelar la realidad industrial y blindar la integridad de los datos. Al conjugar el encapsulamiento estricto con la validación de invariantes de clase bajo un enfoque Fail-Fast, el software impide por diseño la existencia de estados inconsistentes —tales como lotes mal codificados, mediciones con

instrumentos sin calibrar, evaluaciones térmicas manipuladas fuera del umbral de 4.0 °C o alteraciones posteriores al cierre del acta—, garantizando así la trazabilidad sanitaria desde el primer instante de ejecución.

Por otra parte, la migración exitosa del proyecto desde Java hacia TypeScript confirma empíricamente la portabilidad y universalidad de los pilares de la POO. A pesar de las diferencias sintácticas y de plataforma entre la Máquina Virtual de Java (JVM) y el motor de ejecución de Node.js/TypeScript, conceptos estructurales como la abstracción de entidades, el ocultamiento de estado, la sobrecarga o parametrización por defecto de constructores, el polimorfismo de excepciones y la gestión de colecciones dinámicas se mantuvieron inalterados en su esencia lógica, permitiendo adaptar cada regla de negocio a las construcciones idiomáticas propias del lenguaje destino sin perder cohesión ni robustez.

En conclusión, la articulación metódica entre el análisis de dominio (SCQA), el diseño arquitectónico estandarizado en UML y la implementación verificada en múltiples lenguajes consolida una práctica profesional de desarrollo de software. Esta trazabilidad de extremo a extremo asegura que cada línea de código responda directamente a una restricción real del negocio atunero, facilitando el mantenimiento evolutivo, las pruebas automatizadas y la escalabilidad futura del sistema en entornos productivos.